AUTOMAÇÃO COM PYTHON

Apostila prática e original para aprender a automatizar tarefas, arquivos, planilhas, web e processos repetitivos.

Do básico ao projeto final

Python • arquivos • pastas • CSV/JSON • Excel • e-mail • APIs • web • automação de navegador • PyAutoGUI • organização e segurança

Material didático original — elaborado como uma trilha prática de aprendizagem. Não é uma reprodução do material do SlideShare ou de livro protegido.

Sumário

Módulo	Tema
1	Fundamentos da automação
2	Arquivos, pastas e caminhos
3	Leitura e escrita de dados
4	CSV e JSON
5	Planilhas Excel
6	Automação de documentos
7	E-mail e notificações
8	APIs e integração entre sistemas
9	Web scraping e coleta de dados
10	Automação de navegador
11	PyAutoGUI e automação de interface
12	Agendamento e execução de tarefas
13	Tratamento de erros e logs
14	Boas práticas, segurança e manutenção
15	Projeto final: pipeline de automação

1. Fundamentos da automação

O que é automação?

Automação é a utilização de software para executar, de forma consistente, tarefas que antes dependiam de intervenção manual. Em Python, uma automação pode ser simples — como renomear centenas de arquivos — ou envolver vários sistemas, como baixar dados de uma API, transformar os registros, atualizar uma planilha e enviar um relatório.

Como pensar uma automação

Antes de escrever código, descreva o processo em quatro partes: entrada, processamento, saída e regras de exceção. Quanto mais claramente essas partes forem definidas, menor será o risco de criar um script difícil de manter.

Primeiro exemplo

Prática

Escolha uma tarefa repetitiva do seu dia a dia. Escreva: o que inicia o processo, quais dados entram, quais decisões são tomadas, qual é o resultado esperado e o que deve acontecer quando algo der errado.

2. Arquivos, pastas e caminhos

Trabalhando com pathlib

A biblioteca pathlib oferece uma forma moderna de manipular caminhos, arquivos e diretórios, reduzindo problemas causados por diferenças entre Windows, Linux e macOS.

Exemplo de código

```
from pathlib import Path

pasta = Path("entrada")

for arquivo in pasta.iterdir():
    if arquivo.is_file():
        print(arquivo.name, arquivo.suffix)
```

Exemplo: listar arquivos

Exemplo de código

```
from pathlib import Path
import shutil

origem = Path("entrada")
destino = Path("organizados")
destino.mkdir(exist_ok=True)

for arquivo in origem.glob("*.pdf"):
    shutil.move(str(arquivo), destino / arquivo.name)
```

Criando e movendo arquivos

Prática

Crie uma pasta de entrada com arquivos de diferentes extensões. Faça um programa que identifique cada extensão e organize os arquivos em subpastas.

Exercício sugerido: implemente o exemplo sem copiar o código e depois acrescente pelo menos uma validação de erro.

3. Leitura e escrita de dados

Arquivos de texto

Para dados simples, arquivos de texto continuam sendo uma opção útil. O contexto mais importante é definir a codificação, normalmente UTF-8, e usar with para garantir o fechamento correto do arquivo.

Exemplo de código

```
from pathlib import Path

arquivo = Path("dados.txt")

with arquivo.open("r", encoding="utf-8") as f:
    texto = f.read()

print(texto)
```

Exemplo

Exemplo de código

```
linhas_validas = []

for linha in texto.splitlines():
    linha = linha.strip()
    if linha:
        linhas_validas.append(linha)

print(linhas_validas)
```

Processando linhas

4. CSV e JSON

CSV

CSV é adequado para dados tabulares simples. Python possui o módulo csv na biblioteca padrão, permitindo ler e escrever registros sem depender de um banco de dados.

Exemplo de código

```
import json
import csv

with open("clientes.csv", newline="", encoding="utf-8") as f:
    registros = list(csv.DictReader(f))

with open("clientes.json", "w", encoding="utf-8") as f:
    json.dump(registros, f, ensure_ascii=False, indent=2)
```

JSON

JSON é muito comum em APIs e configurações. Ele representa objetos, listas, números, textos e valores booleanos de maneira estruturada.

Exemplo de transformação

Exercício sugerido: implemente o exemplo sem copiar o código e depois acrescente pelo menos uma validação de erro.

5. Planilhas Excel

Por que automatizar planilhas?

Planilhas são um ponto frequente de trabalho manual: copiar dados, consolidar abas, calcular indicadores e gerar relatórios. Bibliotecas como openpyxl permitem criar e modificar arquivos XLSX diretamente.

Exemplo de código

```
from openpyxl import Workbook

wb = Workbook()
ws = wb.active
ws.title = "Vendas"

ws.append(["Produto", "Quantidade", "Valor"])
ws.append(["Teclado", 4, 120.00])
ws.append(["Mouse", 8, 80.00])

wb.save("relatorio.xlsx")
```

Exemplo com openpyxl

Boas práticas

Evite alterar a planilha original sem necessidade. Prefira trabalhar com uma cópia, registre a origem dos dados e valide o resultado antes de distribuir o arquivo.

Exercício sugerido: implemente o exemplo sem copiar o código e depois acrescente pelo menos uma validação de erro.

6. Automação de documentos

Documentos e relatórios

Relatórios podem ser gerados a partir de dados estruturados. A ideia central é separar conteúdo, regras de negócio e apresentação, permitindo atualizar o documento sem reescrever todo o processo.

Estrutura recomendada

Uma automação de relatório pode seguir: coletar → validar → transformar → montar → exportar → revisar → distribuir.

7. E-mail e notificações

Automatizando comunicação

Uma automação pode enviar notificações quando uma rotina termina, quando uma condição é atingida ou quando ocorre uma falha. Em ambientes reais, credenciais não devem ficar escritas diretamente no código.

Exemplo de código

```
import os
import smtplib
from email.message import EmailMessage

msg = EmailMessage()
msg["Subject"] = "Processamento concluído"
msg["From"] = os.environ["EMAIL_REMETENTE"]
msg["To"] = os.environ["EMAIL_DESTINO"]
msg.set_content("A rotina foi concluída com sucesso.")

# A configuração SMTP depende do provedor utilizado.
# Nunca coloque a senha diretamente no código.
```

Exemplo conceitual

8. APIs e integração entre sistemas

O que é uma API?

Uma API permite que um programa converse com outro sistema de forma estruturada. Em Python, requests é uma escolha comum para chamadas HTTP.

Exemplo de código

```
import requests

resposta = requests.get(
    "https://api.exemplo.com/clientes",
    timeout=15
)
resposta.raise_for_status()

dados = resposta.json()
print(dados)
```

GET e tratamento da resposta

Cuidados

Valide códigos de status, trate timeout e erros de conexão, respeite limites de requisição e não exponha tokens ou chaves em logs públicos.

Exercício sugerido: implemente o exemplo sem copiar o código e depois acrescente pelo menos uma validação de erro.

9. Web scraping e coleta de dados

Quando usar scraping

Web scraping pode ser útil quando dados públicos precisam ser coletados de páginas que não oferecem uma API adequada. Antes de coletar, verifique termos de uso, permissões e regras aplicáveis ao site.

Exemplo de código

```
import requests
from bs4 import BeautifulSoup

html = requests.get("https://exemplo.com", timeout=15).text
soup = BeautifulSoup(html, "html.parser")

for titulo in soup.select("h2"):
    print(titulo.get_text(strip=True))
```

BeautifulSoup

Estratégia robusta

Prefira seletores estáveis, valide se os elementos existem e registre quais páginas foram processadas. Para grandes volumes, considere paginação, cache e controle de velocidade.

10. Automação de navegador

Automação de páginas dinâmicas

Quando o conteúdo depende de JavaScript ou exige interação, uma ferramenta de automação de navegador pode ser mais adequada que uma simples requisição HTTP. Playwright e Selenium são opções conhecidas.

Exemplo de código

```
from playwright.sync_api import sync_playwright

with sync_playwright() as p:
    browser = p.chromium.launch(headless=True)
    page = browser.new_page()
    page.goto("https://exemplo.com")
    print(page.title())
    browser.close()
```

Exemplo com Playwright

Estabilidade

Evite depender de posições visuais. Prefira seletores semânticos, IDs estáveis ou atributos definidos para automação. Use esperas explícitas e valide o resultado de cada etapa.

Exercício sugerido: implemente o exemplo sem copiar o código e depois acrescente pelo menos uma validação de erro.

11. PyAutoGUI e automação de interface

Automatizando teclado e mouse

PyAutoGUI permite controlar teclado, mouse e elementos da tela. É útil para aplicações sem integração programática, mas tende a ser mais frágil que uma API ou integração direta.

Exemplo de código

```
import pyautogui
import time

time.sleep(2)
pyautogui.write("texto de teste", interval=0.03)
pyautogui.press("enter")
```

Exemplo

Segurança

Inclua pausas, atalhos para interrupção e testes em ambiente controlado. Nunca execute uma automação destrutiva sem uma forma clara de interrompê-la.

12. Agendamento e execução de tarefas

Rotinas recorrentes

Uma automação torna-se muito mais útil quando pode ser executada sem intervenção. No Windows, o Agendador de Tarefas pode iniciar scripts Python em horários definidos; em Linux, cron é uma alternativa tradicional.

Exemplo de código

```
from datetime import datetime
from pathlib import Path

log = Path("execucoes.log")

with log.open("a", encoding="utf-8") as f:
    f.write(f"Execução: {datetime.now():%Y-%m-%d %H:%M:%S}\n")
```

Script preparado para produção

13. Tratamento de erros e logs

Erros são parte do processo

Uma automação profissional deve considerar arquivos ausentes, dados inválidos, indisponibilidade de rede, credenciais expiradas e mudanças no sistema externo.

Exemplo de código

```
try:
    valor = int(input("Digite um número: "))
    resultado = 100 / valor
except ValueError:
    print("Entrada inválida.")
except ZeroDivisionError:
    print("Não é possível dividir por zero.")
else:
    print(resultado)
```

try/except

Exemplo de código

```
import logging

logging.basicConfig(
    filename="automacao.log",
    level=logging.INFO,
    format="%(asctime)s | %(levelname)s | %(message)s"
)

logging.info("Processamento iniciado")
logging.warning("Registro sem e-mail")
logging.info("Processamento concluído")
```

Logging

Exercício sugerido: implemente o exemplo sem copiar o código e depois acrescente pelo menos uma validação de erro.

14. Boas práticas, segurança e manutenção

Princípios essenciais

Mantenha funções pequenas, nomes claros, configuração separada do código, dependências documentadas e logs úteis. Versione o projeto com Git e teste as etapas críticas.

Segredos e configurações

Use variáveis de ambiente ou um gerenciador de segredos. Nunca publique senhas, tokens, cookies de sessão ou chaves privadas em repositórios.

Checklist de produção

Entrada validada; saída conferida; erros tratados; logs registrados; credenciais protegidas; caminho de recuperação definido; execução idempotente quando possível; documentação mínima disponível.

15. Projeto final: pipeline de automação

Objetivo

Construir uma automação que receba arquivos de uma pasta, valide os registros, transforme os dados, gere um relatório Excel, produza um resumo e registre tudo em log.

Exemplo de código

```
projeto/
├── main.py
├── config.py
├── requirements.txt
├── entrada/
├── saida/
├── logs/
└── src/
    ├── validacao.py
    ├── transformacao.py
    └── relatorio.py
```

Arquitetura

1) Entrada → 2) Validação → 3) Transformação → 4) Relatório → 5) Log → 6) Notificação. Cada etapa deve poder ser testada separadamente.

Exemplo de estrutura

Desafios

Adicione: processamento incremental, arquivos duplicados, relatório de erros, configuração externa, execução agendada e uma notificação quando o processo terminar.

Exercício sugerido: implemente o exemplo sem copiar o código e depois acrescente pelo menos uma validação de erro.

Guia rápido de bibliotecas

pathlib — caminhos, arquivos e diretórios.
csv — leitura e escrita de arquivos tabulares.
json — dados estruturados e configurações.
openpyxl — arquivos Excel XLSX.
requests — chamadas HTTP e APIs.
BeautifulSoup — análise de HTML.
Playwright/Selenium — automação de navegadores.
PyAutoGUI — automação de teclado e mouse.
logging — registros de execução.

Trilha recomendada de estudo

1. Aprenda Python básico e funções. 2. Domine arquivos e estruturas de dados. 3. Automatize tarefas locais. 4. Trabalhe com CSV/JSON/Excel. 5. Integre APIs. 6. Aprenda web scraping responsável. 7. Automatize navegador quando necessário. 8. Adicione logs, testes, configuração e agendamento. 9. Construa projetos completos.

Nota sobre a fonte de inspiração

Esta apostila foi criada de forma original, tomando como referência temática a ideia geral de automação com Python. Ela não reproduz nem transcreve o conteúdo integral do material hospedado no SlideShare.